

System Documentation: Recipe Management Application

Jaehoon Song

May 18, 2026

1 Introduction

This document provides the formal specifications for the Recipe and Ingredient management system in accordance with **ISO/IEC/IEEE 29148:2018** software-requirements practice. It specifies:

- **Backend services:** HTTP endpoints, validation logic, database integration, and response contracts (Javalin REST API).
- **Client application:** Multi-page web UI requirements expressed through UML use-case models, navigation maps, UI element inventories, and interaction (sequence) diagrams.

The system enables user registration and authentication, recipe CRUD and search for all authenticated chefs, and admin-only ingredient taxonomy management.

1.1 Architecture

The system is built utilizing a robust **3-Layer Architecture**, leveraging the **Javalin Framework** for HTTP routing and handling:

- **Controller Layer:** Manages all incoming HTTP requests via Javalin handlers (e.g., `RecipeController`, `AuthenticationController`). It extracts path/query parameters and request bodies, interacts with the Service Layer, and responds with appropriate JSON structures and HTTP status codes.
- **Service Layer:** Encapsulates all business logic and validation rules (e.g., `RecipeService`). It acts as the intermediary between the controllers and DAOs, ensuring data integrity.
- **Data Access Object (DAO) Layer:** Abstracts all direct database interactions using JDBC. It executes CRUD operations against the in-memory H2 database using `ConnectionUtil`.

1.2 Database Entities

Figure 1 shows a compact **Crow's Foot** (IE) style ERD for the primary tables.

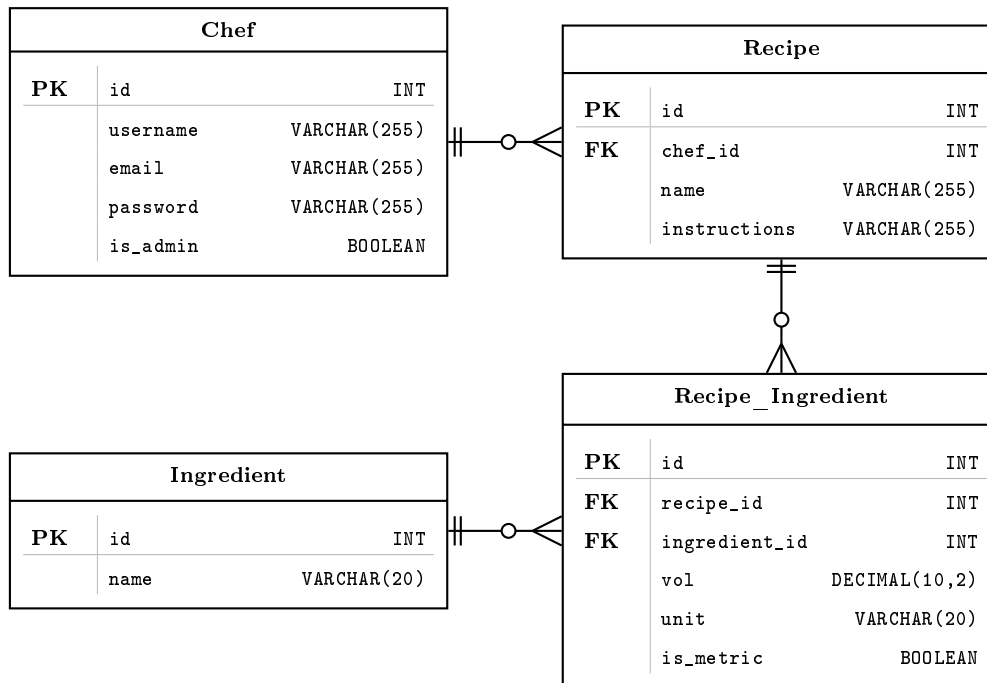


Figure 1: Crow's Foot ERD (Recipe Management System)

Physical mapping (H2). There are four primary tables: CHEF, RECIPE, INGREDIENT, and a join table RECIPE_INGREDIENT. Primary keys are AUTO_INCREMENT INT fields.

1.3 CORS & Security

CORS (Cross-Origin Resource Sharing) is a browser-enforced security mechanism. Before non-simple cross-origin requests (POST, PUT, DELETE), the browser issues a preflight OPTIONS request. The Javalin backend responds globally with:

```
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, Authorization
```

Authorization is performed using Bearer tokens in the Authorization header (Bearer <token>). Protected endpoints return 401 Unauthorized when the token is missing or invalid; the client must alert the user. Ingredient management endpoints additionally require the authenticated chef to have is_admin = true.

2 User Management

2.1 User Registration

A new account is created via a POST request to `/register`.

- **Frontend Behavior:** The frontend must collect `username`, `email`, `password`, and a matching `repeat password`.
- **Backend Handling:** Responds with 201 `Created` with the user details upon success.
- **Error Handling:** If the username already exists, it responds with 409 `Conflict`.

2.2 User Login and Logout

Authentication is verified via a POST request to `/login`.

- **Request Body:** JSON containing `username` and `password`.
- **Successful Response:** Responds with 200 `OK`. Depending on the `Accept` header, it responds with either JSON containing `{"token": "...", "isAdmin": true/false}` or a plaintext string format `"<token> <isAdmin>"`.
- **Error Handling:** Responds with 401 `Unauthorized` if credentials are invalid.

Logout is achieved via a POST request to `/logout` providing the auth token in the `Authorization` header. Responds with 200 `OK` upon successful invalidation of the token.

3 Recipe Management

3.1 Create Recipe

Submit via POST to `/recipes`.

- **Request Body:** JSON representation of a recipe (`name`, `instructions`).
- **Successful Response:** Responds with 201 `Created`. Requires a valid `Bearer` token; otherwise returns 401 `Unauthorized`.

3.2 Retrieve Recipes

List recipes via GET to `/recipes`.

- **Query Params:** Supports pagination (`page`, `pageSize`) and search/filtering (`term`, `name`, `ingredient`).
- **Successful Response:** Responds with 200 `OK` and the list or paginated object of recipes.
- **Error Handling:** Responds with 404 `Not Found` if no recipes match the search criteria.

3.3 Retrieve Recipe by ID

Fetch via GET to `/recipes/{id}`.

- **Successful Response:** Responds with 200 `OK` and the single recipe JSON.
- **Error Handling:** Responds with 404 `Not Found` if the recipe ID does not exist.

3.4 Update Recipe

Update via PUT to `/recipes/{id}`.

- **Request Body:** JSON representation of the updated recipe.
- **Successful Response:** Responds with 200 OK and the updated recipe JSON.
- **Error Handling:** Responds with 404 Not Found if the recipe doesn't exist.

3.5 Delete Recipe

Delete via DELETE to `/recipes/{id}`.

- **Successful Response:** Responds with 200 OK.
- **Error Handling:** Responds with 404 Not Found if the recipe didn't exist.

4 Ingredient Management (Admin Only)

4.1 Create Ingredient

Submit via POST to `/ingredients`.

- **Request Body:** JSON representation of an ingredient (`name`).
- **Successful Response:** Responds with 201 Created.

4.2 Retrieve Ingredients

List ingredients via GET to `/ingredients`.

- **Query Params:** Supports pagination (`page`, `pageSize`) and filtering (`term`).
- **Successful Response:** Responds with 200 OK and the list or paginated object.

4.3 Retrieve Ingredient by ID

Fetch via GET to `/ingredients/{id}`.

- **Successful Response:** Responds with 200 OK and the single ingredient JSON, or 404 Not Found.

4.4 Update Ingredient

Update via PUT to `/ingredients/{id}`.

- **Successful Response:** Responds with 204 No Content on successful update.

4.5 Delete Ingredient

Delete via DELETE to `/ingredients/{id}`.

- **Successful Response:** Responds with 204 No Content.

5 System Context and Actors

5.1 Purpose and Scope

The deployable system comprises a **monolithic Javalin REST API** (port 8081) co-located with a static-file server, and a **vanilla HTML/CSS/JavaScript** multi-page application (MPA). Scope includes all user-visible flows described in the project README: registration, login/logout, recipe management, and admin ingredient management.

5.2 Actors and User Roles

Actor	Role	Capabilities
Guest	Unauthenticated visitor	Register account; log in
Chef	Authenticated user	Create, read, update, delete, and search own recipes; log out
Admin Chef	Chef with <code>is_admin=true</code>	All Chef capabilities plus ingredient CRUD via admin UI

Table 1: System actors (ISO/IEC/IEEE 29148 user characteristics)

5.3 System Context Diagram

Figure 2 situates the application within its runtime environment (C4 Level 1 / UML deployment context).

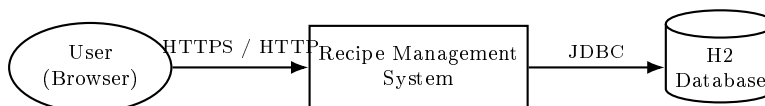


Figure 2: System context: browser client, Javalin server, in-memory H2

6 Client Application Architecture

6.1 Technology Stack

The frontend is built strictly with **Vanilla HTML, CSS, and JavaScript**—no SPA framework. Each domain feature is a separate HTML page with a companion script, following a **Multi-Page Application (MPA)** pattern.

6.2 Module Layout

Source files reside under `src/main/resources/public/frontend/`:

```
frontend/  
  login/      # Authentication entry point  
  register/   # User creation  
  recipe/     # Primary dashboard and recipe CRUD  
  ingredients/ # Admin-only ingredient management
```

6.3 Runtime and Deployment

- **Base URL:** `http://localhost:8081/` serves both the REST API and static assets.
- **Test isolation:** Do not run automated Selenium tests concurrently with a manual backend instance on the same port.
- **Presentation:** CSS is optional; markup must expose the DOM element IDs specified in Section 8.

6.4 Client-Side State Model

Ephemeral session state is stored in `sessionStorage`:

Key	Type	Purpose
<code>auth-token</code>	string	Bearer token (JWT/UUID) from successful login
<code>is-admin</code>	string (<code>"true"/"false"</code>)	RBAC flag; controls visibility of admin navigation

Table 2: Browser session storage contract

6.5 API Client Integration (Fetch API)

All network I/O uses the native **Fetch API**:

- JSON requests set `Content-Type: application/json`.
- Protected calls include `Authorization: Bearer <auth-token>`.
- Flow control checks `response.ok` and explicit status codes (201, 409, 404, etc.).
- CORS preflight is implicit; see Section 1.3.

6.6 Navigation Map

Figure 3 defines allowable page transitions. Unauthenticated users access `register` and `login`; authenticated users land on `recipe`; admins may navigate to `ingredients`.

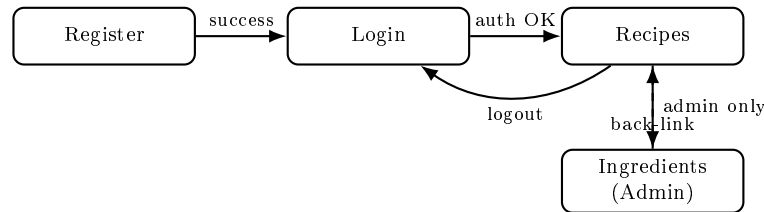


Figure 3: MPA navigation / screen-flow diagram

7 Functional Requirements: Use Case Model

7.1 Use Case Diagram

Figure 4 follows UML use-case notation: actors outside the system boundary; ovals denote use cases grouped by package.

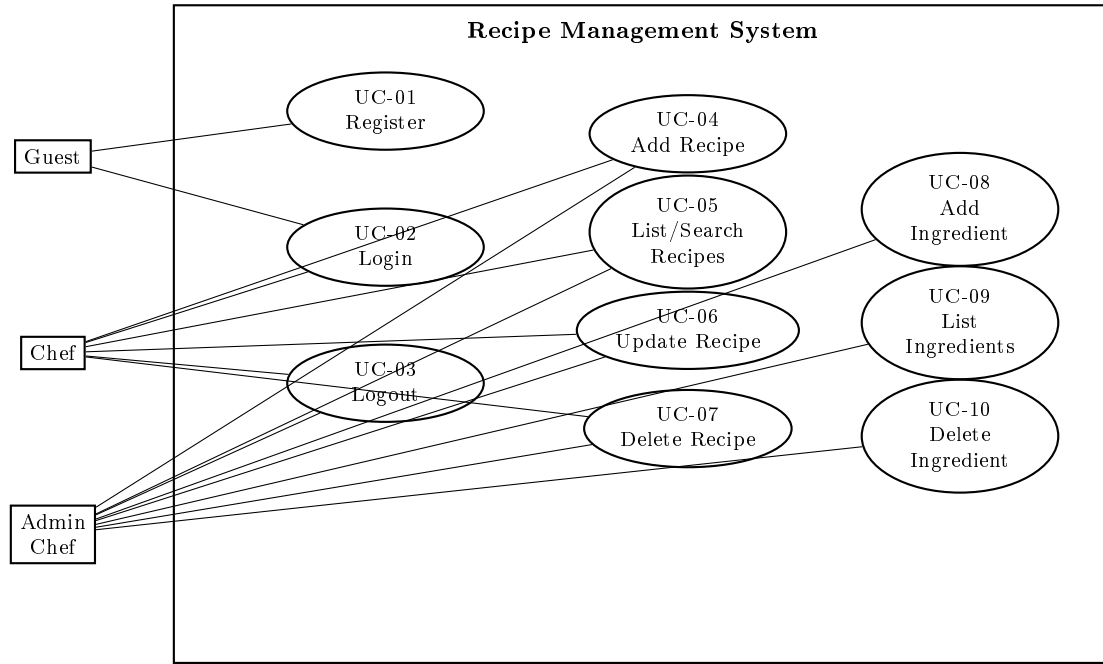


Figure 4: UML use case diagram (client-visible functionality)

7.2 Use Case Catalogue

ID	Name	Primary Actor	Maps to README
UC-01	Register Account	Guest	User Registration
UC-02	Authenticate	Guest/Chef	User Login
UC-03	Log Out	Chef	Logout (recipe page)
UC-04	Add Recipe	Chef	Recipe Management
UC-05	List/Search Recipes	Chef	Recipe Management
UC-06	Update Recipe	Chef	Recipe Management
UC-07	Delete Recipe	Chef	Recipe Management
UC-08	Add Ingredient	Admin Chef	Ingredient Management
UC-09	List Ingredients	Admin Chef	Ingredient Management
UC-10	Delete Ingredient	Admin Chef	Ingredient Management

Table 3: Use case traceability to project requirements

7.3 Use Case Specifications

The following narratives follow the standard UML use-case specification template (brief description, pre/postconditions, basic and alternative flows).

7.3.1 UC-01: Register Account

Location: frontend/register/ (register-page.html, register-page.js).

Preconditions: Guest is on the registration page.

Basic Flow:

1. Guest enters `username`, `email`, `password`, and `repeat password`.
2. Guest clicks `register-button`.
3. System validates all fields are non-empty and passwords match.
4. System sends `POST /register` with JSON body `{username, email, password}`.
5. System receives `201 Created` and redirects to the login page.

Alternative Flows:

- **3a** Validation fails → display alert; remain on page.
- **5a** `409 Conflict` → alert duplicate user/email.
- **5b** Other error or network failure → alert generic or network error.

7.3.2 UC-02: Authenticate

Location: `frontend/login/`.

Basic Flow:

1. Guest enters `username` and `password`; clicks `login-button`.
2. System sends `POST /login` with JSON credentials.
3. On `200 OK`, system parses token and admin flag (JSON `{token, isAdmin}` or plaintext "`<token> <isAdmin>`").
4. System stores `auth-token` and `is-admin` in `sessionStorage`.
5. System redirects to `frontend/recipe/recipe-page.html`.

Alternative Flows: `401` → alert invalid credentials; empty fields → alert before request.

7.3.3 UC-03: Log Out

Location: `frontend/recipe/recipe-page.js` (`processLogout`).

Basic Flow:

1. Chef clicks `logout-button` (visible when `auth-token` exists).
2. System sends `POST /logout` with Bearer token.
3. On `200 OK`, system clears `sessionStorage` and redirects to login.

7.3.4 UC-04 through UC-07: Recipe CRUD

Location: `frontend/recipe/`. All flows require a valid Bearer token. After mutating operations, the client refreshes the in-memory `recipes` array and re-renders `recipe-list`.

Name-to-ID resolution (client pattern): Update and delete forms accept a **recipe name**. The client resolves the REST path parameter by case-insensitive match against the locally cached list from `GET /recipes`, then calls `PUT /recipes/{id}` or `DELETE /recipes/{id}`. If no match exists, alert "Recipe not found in list!"

7.3.5 UC-08 through UC-10: Ingredient Management (Admin)

Location: `frontend/ingredients/`. Accessible when `sessionStorage.is-admin === "true"`. Delete resolves ingredient **name** to **id** via local cache (same pattern as recipes). Successful delete accepts `204 No Content` or `200 OK`.

UC	UI Trigger	HTTP	Success
UC-04	add-recipe-submit-input	POST /recipes	201, refresh list
UC-05	page load / search-button	GET /recipes or ?name=	200/404 → render
UC-06	update-recipe-submit-input	PUT /recipes/{id}	200, refresh
UC-07	delete-recipe-submit-input	DELETE /recipes/{id}	200, refresh

Table 4: Recipe use cases and REST mapping

8 User Interface Specifications

Each subsection lists mandatory DOM identifiers (wireframe-level specification per ISO/IEC/IEEE 29148 Section 8) and required JavaScript behaviors.

8.1 Registration Page

Element ID	Control	Requirement
username-input	text	Required; sent to API
email-input	email	Required; sent to API
password-input	password	Required; sent to API
repeat-password-input	password	Client-only confirmation
register-button	button	Invokes <code>processRegistration()</code>

Table 5: Registration page UI inventory

8.2 Login Page

Element ID	Control	Requirement
login-input	text	Username for POST /login
password-input	password	Password for POST /login
login-button	button	Invokes <code>processLogin()</code>
logout-button	button	Hidden by default; optional testing aid

Table 6: Login page UI inventory

8.3 Recipe Dashboard

Page-load behavior: On `DOMContentLoaded`, call `getRecipes()`, show admin-link via admin check (equivalent to `displayAdminLink()`), wire event handlers, and conditionally display `logout-button`.

Required functions: `getRecipes()`, `refreshRecipeList()`, `addRecipe()`, `updateRecipe()`, `deleteRecipe()`, `searchRecipes()`, `processLogout()`.

8.4 Ingredient Management Page (Admin)

Page-load behavior: Call `getIngredients()`. Verify auth-token and admin status via `sessionStorage` before allowing mutations.

Required functions: `getIngredients()`, `addIngredient()`, `deleteIngredient()`, `refreshIngredientList()`.

9 Client–Server Interaction Models

9.1 Sequence Diagram: User Registration

Figure 5 models UC-01 as a UML interaction diagram.

Element ID	Control	Requirement
admin-link	anchor	Hidden initially; shown when <code>is-admin</code> is <code>true</code> ; links to ingredients page
logout-button	button	Hidden until <code>auth-token</code> present
search-input, search-button	text, button	Invokes <code>searchRecipes()</code>
recipe-list		Populated by <code>refreshRecipeList()</code>
add-recipe-name-input	text	Add form
add-recipe-instructions-input	textarea	Add form
add-recipe-submit-input	button	Invokes <code>addRecipe()</code>
update-recipe-name-input	text	Name lookup for update
update-recipe-instructions-input	textarea	New instructions
update-recipe-submit-input	button	Invokes <code>updateRecipe()</code>
delete-recipe-name-input	text	Name lookup for delete
delete-recipe-submit-input	button	Invokes <code>deleteRecipe()</code>

Table 7: Recipe page UI inventory

Element ID	Control	Requirement
back-link	anchor	Return to recipe dashboard
ingredient-list		Rendered by <code>refreshIngredientList()</code>
add-ingredient-name-input	text	Add form
add-ingredient-submit-button	button	Invokes <code>addIngredient()</code>
delete-ingredient-name-input	text	Name lookup for delete
delete-ingredient-submit-button	button	Invokes <code>deleteIngredient()</code>

Table 8: Ingredient page UI inventory

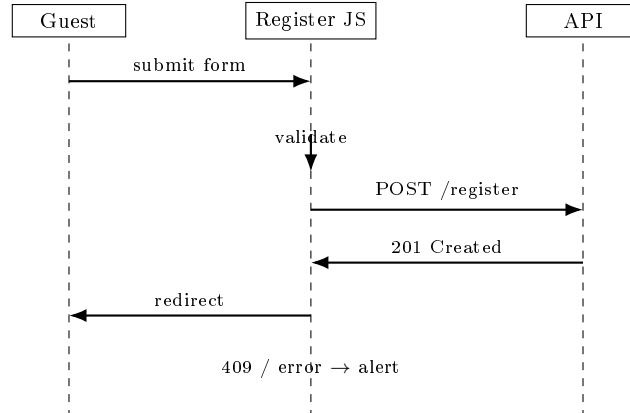


Figure 5: Sequence diagram: registration (UC-01)

9.2 Sequence Diagram: Login and Session Establishment

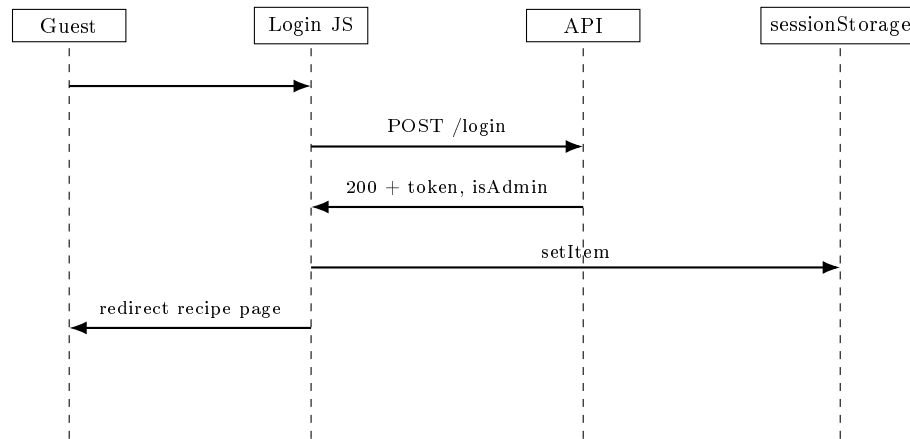


Figure 6: Sequence diagram: authentication (UC-02)

9.3 Activity Diagram: Update Recipe by Name

The client resolves the REST identifier before invoking the API (UML activity semantics):

1. Read `update-recipe-name-input` and `update-recipe-instructions-input`.
2. If either empty → alert and stop.
3. Search `recipes[]` for case-insensitive name match.
4. If not found → alert and stop.
5. PUT `/recipes/{id}` with updated instructions; on 200 refresh list.

10 Non-Functional Requirements (Client)

- **Usability:** Use browser `alert()` for validation and error feedback unless otherwise specified.
- **Security:** No credentials persisted beyond `sessionStorage`; protected pages must attach Bearer tokens.
- **Compatibility:** Modern browsers with Fetch and `sessionStorage` support.
- **Maintainability:** One HTML/JS pair per feature directory; shared `BASE_URL` = "`http://localhost:8081`".

11 External References

- MDN Fetch API: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
- MDN CORS: <https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>
- ISO/IEC/IEEE 29148:2018 — Systems and software engineering — Life cycle processes — Requirements engineering